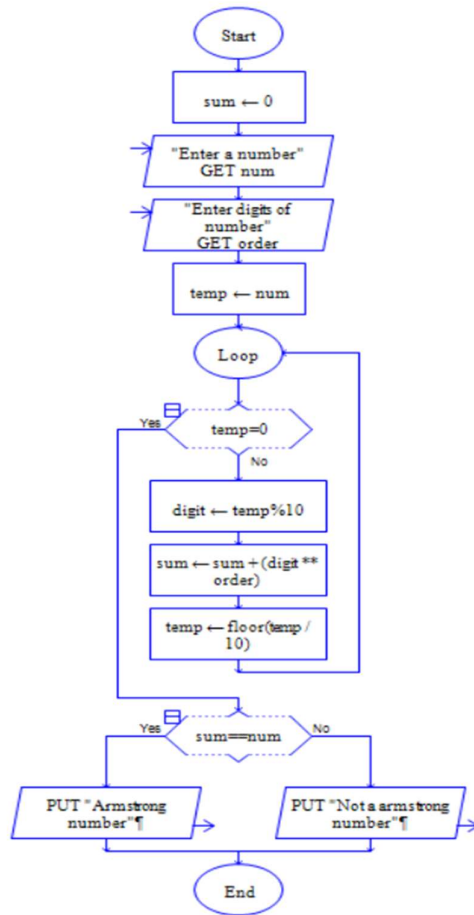


EX01- ARMSTRONG NUMBER

AIM:

To design and implement a RAPTOR flowchart to check whether a given number is an Armstrong number or not.

FLOWCHART:



SAMPLE INPUT:

153

SAMPLE OUTPUT:

Armstrong number

RESULT:

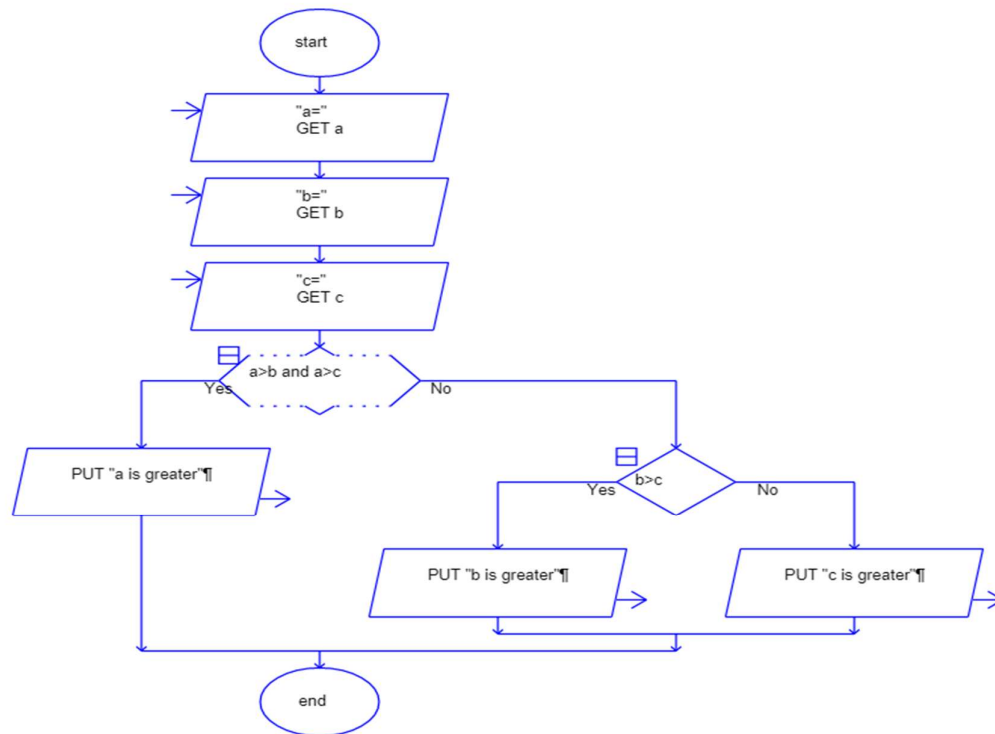
Thus, the RAPTOR flowchart was designed and executed successfully to determine whether the given number is an Armstrong number or not.

EX02- GREATEST OF THREE NUMBERS

AIM:

To design and implement a RAPTOR flowchart to find the largest among three given numbers.

FLOWCHART:



SAMPLE INPUT:

A = 25 , B = 18 , C = 40

SAMPLE OUTPUT:

40 is greater

RESULT:

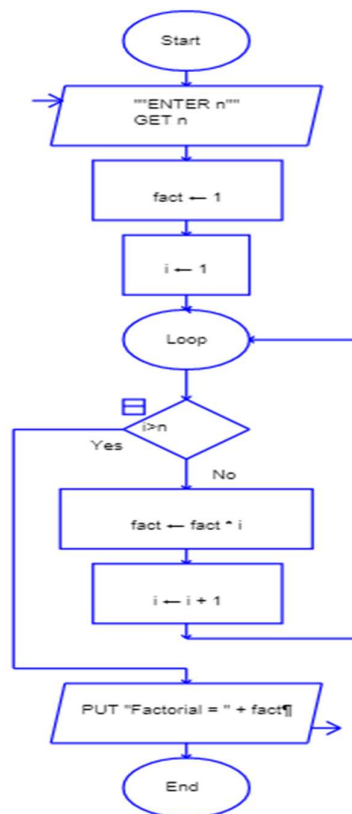
Thus, the RAPTOR flowchart was designed and executed successfully to find the maximum among three given numbers.

EX03- FACTORIAL OF A NUMBER

AIM:

To design and implement a RAPTOR flowchart to find the factorial of a given number.

FLOWCHART:



SAMPLE INPUT:

5

SAMPLE OUTPUT:

Factorial = 120

RESULT:

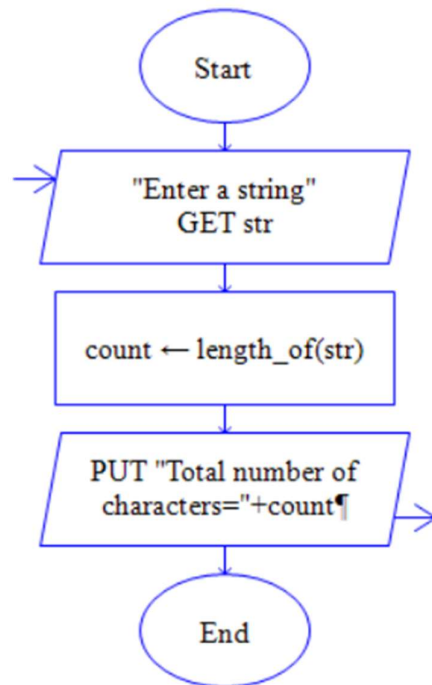
Thus, the RAPTOR flowchart was executed successfully and the factorial of the given number was displayed correctly.

EX04- STRING HANDLING – FINDING THE TOTAL NUMBER OF CHARACTERS IN A STRING

AIM:

To design and implement a RAPTOR flowchart to find and display the total number of characters in a given string.

FLOWCHART:



SAMPLE INPUT:

COMPUTER

SAMPLE OUTPUT:

Total number of characters= 8

RESULT:

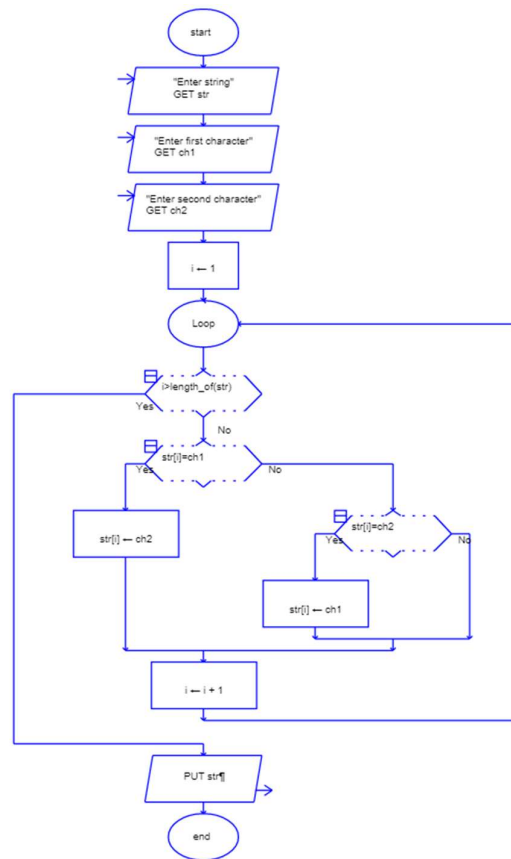
Thus, the RAPTOR flowchart was executed successfully and the total number of characters in the given string was displayed correctly.

EX05- FINDING THE LEXICOGRAPHICALLY SMALLEST STRING BY SWAPPING ANY TWO CHARACTERS USING RAPTOR

AIM:

To design a RAPTOR flowchart to find the lexicographically smallest string after swapping any two characters at most once.

FLOWCHART:



SAMPLE INPUT:

abca

SAMPLE OUTPUT:

bacb

RESULT:

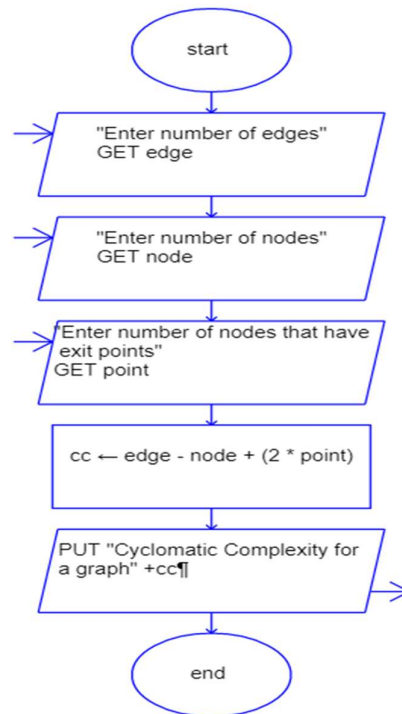
Thus, the RAPTOR flowchart was successfully designed and executed to find the lexicographically smallest string.

EX06- CALCULATION OF CYCLOMATIC COMPLEXITY OF A FLOW GRAPH

AIM:

To design and implement a RAPTOR flowchart to calculate the Cyclomatic Complexity of a flow graph using the given number of edges, nodes, and predicate nodes.

FLOWCHART:



SAMPLE INPUT:

Edge=17, Node=14, Point=5

SAMPLE OUTPUT:

Cyclomatic Complexity for a graph=14

RESULT:

Thus, the RAPTOR flowchart was successfully executed, and the Cyclomatic Complexity was calculated as **14**.